

Втор циклус

Системи за откривање на мрежни напади базирани на вештачка интелигенција

(AI based network intrusion detection systems)

Проект по

Дигитална трансформација и вештачка интелигенција

Ментор

Проф. д-р Ристевски Благој

Изработил

Сара Карачанакова 5/25/II

Битола, декември 2025 година

СОДРЖИНА

АПСТРАКТ.....	5
ЛИСТА НА КОРИСТЕНИ ОЗНАКИ.....	3
СПИСОК НА СЛИКИ.....	3
СПИСОК НА ТАБЕЛИ.....	3
СПИСОК НА СКРАТЕНИЦИ.....	4
1. ВОВЕД.....	6
2. МОМЕНТАЛНА СОСТОЈБА (State of the Art).....	7
2.1. Системи базирани на потписи.....	7
2.2. Системи базирани на аномалии.....	7
3. Теоретска основа и објаснување на проектот.....	8
3.1. Опис на NSL-KDD множеството податоци.....	8
3.2. Зошто е потребно да се именуваат колоните?.....	9
3.3. Претпроцесирање (Feature Engineering).....	9
3.4. Машинско учење – користени модели.....	9
3.5. Споредба на моделите.....	10
3.6. Напредни техники.....	10
4. Код.....	12
4.1. Вчитување на податоците.....	12
4.2. Поделба на тренинг и тест множество.....	12
4.3. Претпроцесирање на податоците.....	13
4.4. Decision Tree модел.....	13
4.5. Random Forest модел.....	13
4.6. Logistic Regression модел.....	14
4.7. Neural Network (MLP) модел.....	14
4.8. Евалуација на моделите.....	14
5. Резултати.....	15
5.1. Класификациски резултати (бинарна класификација).....	15
5.2. Матрица на конфузија.....	16
5.3. Важност на карактеристиките (Feature Importance).....	18
5.4. ROC-крива и AUC-анализа.....	19
5.5. Многукласна класификација.....	21
5.6. Пресметковна сложеност.....	23
6. Архитектура на решението и локална примена.....	25
7. Заклучок.....	27
8. Идни подобрувања.....	29
9. Валидација на пристапот врз друго множество податоци (UNSW-NB15).....	31
9.1. Опис на UNSW-NB15.....	31
9.2. Прилагодување на pipeline-от.....	32
9.3. Резултати.....	32
9.4. Споредба со NSL-KDD и заклучок од валидацијата.....	35
10. ЛИТЕРАТУРА.....	37
11. Прилози.....	38

ЛИСТА НА КОРИСТЕНИ ОЗНАКИ

Ознака	Значење на македонски јазик	Значење на англиски јазик
AI	Вештачка интелигенција	Artificial Intelligence
IDS	Систем за детекција на упади	Intrusion Detection System
MLP	Повеќеслоен перцептрон	Multi-Layer Perceptron
ROC/AUC	Работна карактеристика на приемникот / Површина под кривата	Receiver Operating Characteristic / Area Under the Curve
TP / TN	Точно позитивно / Точно негативно	True Positive / True Negative
FP / FN	Погрешно позитивно / Погрешно негативно	False Positive / False Negative
ONE	Еднократно бинарно кодирање	One-Hot Encoding

СПИСОК НА СЛИКИ

Слика бр.	Наслов на сликата
Слика 1	Распределба на класите во тест-множеството (класна нерамнотежа)
Слика 2	Работен тек на решението: тренирање во Google Colab и локално тестирање
Слика 3	Тек на процесот на детекција — од конекција до одлука
Слика 4	Матрица на конфузија — Decision Tree
Слика 5	Матрица на конфузија — Random Forest
Слика 6	Матрица на конфузија — Logistic Regression
Слика 7	Матрица на конфузија — MLP Neural Network
Слика 8	Топ 20 најважни карактеристики (Random Forest, бинарно)
Слика 9	ROC-криви за бинарната класификација
Слика 10	Матрица на конфузија — Random Forest (многукласно)
Слика 11	Распределба на класите во тест-множеството (UNSW-NB15)
Слика 12	ROC-криви за бинарната класификација (UNSW-NB15)
Слика 13	Матрица на конфузија — Random Forest, бинарно (UNSW-NB15)
Слика 14	Матрица на конфузија — Random Forest, многукласно (UNSW-NB15)
Слика 15	Топ 20 најважни карактеристики (UNSW-NB15)

СПИСОК НА ТАБЕЛИ

Табела бр.	Наслов на табелата
Табела 1	Групирање на поединечните напади во пет целни класи
Табела 2	Тематски групи на 41-те атрибути на NSL-KDD
Табела 3	Преглед на користените модели и нивните основни поставки
Табела 4	Резултати од бинарната класификација
Табела 5	Резултати од многукласната класификација
Табела 6	Точност по класа — Random Forest (многукласно)
Табела 7	Пресметковна сложеност — времиња на тренирање
Табела 8	Резултати од бинарната класификација (UNSW-NB15)
Табела 9	Резултати од многукласната класификација (UNSW-NB15)
Табела 10	Точност по класа — Random Forest, многукласно (UNSW-NB15)
Табела 11	Споредба на клучните наоди меѓу NSL-KDD и UNSW-NB15

СПИСОК НА СКРАТЕНИЦИ

На англиски јазик	На македонски јазик
ETL — Extract, Transform, Load	Извлечи, трансформирај, вчитај
DoS — Denial of Service	Одбивање на услуга
R2L — Remote to Local	Одалечено до локално
U2R — User to Root	Од корисник до администратор
OHE — One-Hot Encoding	Еднократно бинарно кодирање
SMOTE — Synthetic Minority Over-sampling Technique	Синтетичко надополнување на малцинските класи
CNN — Convolutional Neural Network	Конволуциска невронска мрежа
LSTM — Long Short-Term Memory	Долга краткорочна меморија (тип рекурентна мрежа)

АПСТРАКТ

Овој проект разработува интелигентен систем за детекција на мрежни напади (Intrusion Detection System – IDS) базиран на машинско учење. Проблемот што се обработува е автоматско разликување помеѓу нормален и нападнат мрежен сообраќај, со цел навремено откривање на потенцијални закани. За реализација на системот е употребено множеството податоци NSL-KDD, кое беше соодветно обработено преку кодирање на категоријалните атрибути, стандардизација на нумеричките атрибути и дефинирање бинарна целна променлива.

Во проектот се имплементирани и споредени четири модели на машинско учење: дрво на одлучување (Decision Tree), случајна шума (Random Forest), логистичка регресија (Logistic Regression) и невронска мрежа од типот повеќеслоен перцептрон (MLP Neural Network). Преку споредбена анализа на метриките точност (accuracy), прецизност (precision), одзив (recall), F1-мерка и ROC/AUC-кривите, се утврди дека Random Forest и MLP Neural Network постигнуваат најдобри перформанси, со највисока моќ за детекција на нападнат сообраќај. Дополнително, истите модели се обучени и во многукласен режим, во кој нападите се распределуваат во четирите познати фамилии (DoS, Probe, R2L и U2R), а анализата на важноста на карактеристиките со Random Forest ги идентификува атрибутите што најсилно придонесуваат кон откривањето напади.

Резултатот е функционален и повторно употреблив модел на IDS — моделите се тренираат еднаш во облак (Google Colab) и потоа локално се вчитуваат и применуваат врз нови податоци без повторно тренирање. Ова е првиот чекор кон систем што може да се прошири за реално-временно мониторирање на мрежен сообраќај и понатамошна примена во системи за сајбер-безбедност.

1. ВОВЕД

Современите компјутерски мрежи се постојано изложени на различни типови сајбер-напади, како што се напади за оневозможување услуга (DoS), извидувачки скенирања (Probe), обиди за неовластен далечински пристап (R2L) и обиди за стекнување администраторски привилегии од обична корисничка сметка (U2R). Традиционалните механизми на заштита — статички firewall-правила или рачно прегледување логови — честопати не се доволни за навремено препознавање и спречување на вакви закани, бидејќи нападите брзо еволуираат, а обемот на сообраќајот го прави рачното следење практично неизводливо во реално време.

Поради тоа, последниве години сè повеќе се развиваат системи за детекција на мрежни напади базирани на вештачка интелигенција (AI-based Intrusion Detection Systems), кои автоматски препознаваат сомнително однесување во мрежниот сообраќај. Во овој проект е развиен ваков систем со примена на машинско учење, со цел да се изгради модел кој самостојно ги класифицира мрежните записи како нормални или напад, а дополнително и по тип на напад.

За тренирање и евалуација е користено множеството податоци NSL-KDD, едно од најпознатите и најчесто користените множества во доменот на мрежна безбедност. На тој начин, проектот овозможува практична демонстрација на тоа како современите алгоритми на машинско учење можат да се применат во реални сценарија на сајбер-безбедност, а истовремено покажува и цел работен тек — од сурови податоци, преку тренирање и зачувување на модели, до нивна повторна употреба врз нови податоци без повторно тренирање.

2. МОМЕНТАЛНА СОСТОЈБА (State of the Art)

Системите за детекција на мрежни напади денес најчесто се групираат во два пристапа, кои во голема мера се надополнуваат еден со друг.

2.1. Системи базирани на потписи

Овие системи ги детектираат нападите според однапред познати обрасци (signature-based detection). Тие се брзи и прецизни за веќе документирани закани, но не можат да детектираат нови, непознати напади (zero-day), бидејќи детекцијата зависи целосно од постоење соодветен потпис во базата.

2.2. Системи базирани на аномалии

Наспроти нив, anomaly-based системите користат статистички и модели на машинско учење за да откријат отстапување од нормалното однесување во мрежата, што им овозможува да препознаат и напади што претходно не биле видени. Во научната литература, последната деценија е доминирана токму од овој втор пристап, а особено од модели на машинско и длабоко учење, бидејќи тие можат да обработуваат големи количини мрежни податоци, автоматски да ги извлекуваат релевантните карактеристики и да постигнат висока точност при класификација во реално време [2].

Меѓу најкористените модели во оваа област се дрвата на одлучување (брзи и лесно интерпретабилни), случајната шума (ансамбл модел кој дава постабилни резултати), логистичката регресија (едноставен, но ефикасен линеарен класификатор) и невронските мрежи од типот повеќеслоен перцептрон, кои можат да фатат посложени, нелинеарни односи во податоците. Истражувањата покажуваат дека комбинацијата од добро дефиниран процес на подготовка на податоците, соодветно означено множество и правилен избор на модел може да доведе до точност над 95%, што го прави овој пристап привлечен и за практична примена, а не само за академско истражување.

3. Теоретска основа и објаснување на проектот

3.1. Опис на NSL-KDD множеството податоци

NSL-KDD [1] е подобрена верзија на постарото KDD Cup '99 множество, од кое се отстранети голем број дупликати и намалена е пристрасноста кон најчестите записи, што го прави поверодостојно за објективна споредба на алгоритми. Секој запис опишува една мрежна конекција преку 41 атрибут, придружен со ознака за типот на сообраќајот (label). Атрибутите можат смислено да се поделат во неколку тематски групи, прикажани во Табела 2: основни карактеристики на самата конекција, содржински обележја што укажуваат на сомнителни дејства во рамките на сесијата, сообраќајни статистики пресметани врз краток временски прозорец и статистики врз конекциите кон истиот домаќин. Три од атрибутите се категорични по природа — `protocol_type`, `service` и `flag` — додека сите останати се нумерички; оваа поделба директно ја одредува стратегијата на претпроцесирање опишана во делот 3.3.

Поединечните типови напади во множеството се групираат во четири фамилии — DoS, Probe, R2L и U2R — според нивната природа, како што е прикажано во Табела 1. За бинарната класификација сите четири фамилии се спојуваат во една класа „напад“, наспроти класата „нормален сообраќај“.

Категорија	Опис	Примери на поединечни напади
Normal	Легитимен мрежен сообраќај	normal
DoS	Преоптоварување и оневозможување на услугата	neptune, smurf, back, teardrop, pod, land
Probe	Извидување и скенирање на мрежата	satan, ipsweep, nmap, portsweep, mscan
R2L	Неовластен далечински пристап до локална сметка	guess_passwd, ftp_write, imap, warezmaster
U2R	Стекнување администраторски привилегии од обична сметка	buffer_overflow, rootkit, loadmodule, perl

Табела 1. Групирање на поединечните напади во пет целни класи.

Група атрибути	Што опишува	Примери
Основни	Карактеристики на самата конекција	duration, protocol_type, service, flag, src_bytes
Содржински	Сомнителни дејства во рамките на сесијата	hot, num_failed_logins, logged_in, root_shell
Сообраќајни	Статистики врз краток временски прозорец	count, srv_count, error_rate, same_srv_rate

Група атрибути	Што опишува	Примери
По домаќин	Статистики врз конекции кон ист домаќин	dst_host_count, dst_host_srv_count, dst_host_serror_rate

Табела 2. Тематски групи на 41-те атрибути. Атрибутите `protocol_type`, `service` и `flag` се категоријни, а останатите нумерички.

3.2. Зошто е потребно да се именуваат колоните?

Оригинаалното множество податоци нема заглавје (header), односно нема имиња на колоните. Без нивно експлицитно именување, Pandas не може да ги идентификува карактеристиките по значење, машинското учење не знае кои колони се категоријални а кои нумерички, а ColumnTransformer не може да оддели one-hot кодирање од стандардизација. Со именување на колоните — според официјалната шема на NSL-KDD — се обезбедува структуриран влез погоден за автоматска обработка.

3.3. Претпроцесирање (Feature Engineering)

Категоријалните атрибути (`protocol_type`, `service`, `flag`) се трансформираат со `OneHotEncoder(handle_unknown='ignore')`, со што секоја можна вредност се претвора во посебно бинарно обележје, а моделот безбедно ги толерира и вредности што не се појавиле при тренирањето. Сите останати, нумерички атрибути се стандардизираат со `StandardScaler`, бидејќи нивните вредности се движат во драстично различни опсези — од делови од секундата до милиони бајти — што инаку би довело алгоритмите неоправдано да им придаваат поголема важност на атрибутите со поголеми бројки. Целото претпроцесирање е спакувано заедно со моделот во единствен `sklearn Pipeline` [3] (преку `ColumnTransformer`), со што истата логика на трансформација автоматски се применува и при тренирањето и подоцна, при предвидувањето врз нови податоци — со тоа целосно се отстранува ризикот од несовпаѓање меѓу двете фази, честа причина за тивки грешки во вакви системи.

3.4. Машинско учење — користени модели

Проектот опфаќа четири модели со различна сложеност и логика на учење, чиј преглед е даден во Табела 3. Дрвото на одлучување (Decision Tree) нуди едноставен и лесно толкувачки модел кој служи и како добра појдовна точка. Случајната шума (Random Forest) надградува врз него така што комбинира голем број дрва и со тоа постигнува поголема стабилност и отпорност на преобучување. Логистичката регресија (Logistic Regression) претставува едноставен линеарен модел кој дава корисна основа за споредба, додека повеќеслојниот перцептрон (MLP Neural Network) воведува

нелинеарно учење преку невронска мрежа. Секој модел се тренира со ист pipeline и со стандардни/подразбирани хиперпараметри како почетна точка — систематско пребарување на подобри хиперпараметри (GridSearchCV) е предвидено како насока за понатамошна работа во делот 7.2, наместо да се прави рачно на пробен принцип.

Модел	Тип	Клучни поставки
Decision Tree (дрво на одлучување)	Дрво на одлучување	стандардни параметри, фиксно семе (random_state=42)
Random Forest (случајна шума)	Ансамбл од дрва	200 дрва, паралелно учење
Logistic Regression (логистичка регресија)	Линеарен модел	до 1000 итерации
MLP Neural Network (повеќеслоен перцептрон)	Плитка невронска мрежа	скриени слоеви 128 и 64

Табела 3. Преглед на користените модели и нивните основни поставки.

Освен наведените четири модели кои се системски евалуирани во делот 5, во склоп на проектот е изградена и подлабока невронска мрежа во Keras [4] (128 → Dropout 0.3 → 64 → излезен слој), како дополнителен, истражувачки модел за проверка дали поголем капацитет носи подобрување. Оваа мрежа е зачувана заедно со останатите модели, но не е вклучена во главната споредбена анализа, бидејќи бара посебна TensorFlow околина за евалуација.

3.5. Споредба на моделите

Со користење на ист сет услови и претпроцесирање, во делот 5 се покажува дека Random Forest и MLP Neural Network најчесто даваат највисока моќ за разликување на класите (највисок AUC), логистичката регресија е највизбрз модел, но со најслаба способност да ги моделира нелинеарните обрасци во сообраќајот, додека дрвото на одлучување останува најдобро за брза интерпретација на одлуките, но со поголема склоност кон преобучување.

3.6. Напредни техники

Покрај основната бинарна класификација, проектот вклучува и повеќе напредни елементи: многукласна класификација според типот на нападот (детално во делот 5.5), анализа на важноста на карактеристиките со Random Forest, ROC/AUC-анализа за сите бинарни модели, како и зачувување и повторно вчитување на веќе истренираните модели со joblib, што овозможува локална примена без повторно тренирање. Целиот

процес на носење одлука за една нова конекција — од влезните атрибути, преку претпроцесирањето и алгоритмот на вештачка интелигенција, до конечниот заклучок дали станува збор за напад — е прикажан на Слика 3.



Слика 3. Тек на процесот на детекција — од нова конекција, преку претпроцесирање и алгоритам на вештачка интелигенција, до одлука дали има напад и од кој тип.

Вреди да се истакне дека, за разлика од тренирањето, ова заклучување е детерминистичко и многу брзо — нема учење, туку само примена на веќе изградениот модел, па одлуката за секоја конекција се носи речиси моментално, што го прави процесот погоден за вградување во систем што би работел блиску до реално време.

4. Код

4.1. Вчитување на податоците

```
import pandas as pd

col_names = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes",
    "land", "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root",
    "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count", "serror_rate",
    "srv_serror_rate", "error_rate", "srv_rerror_rate", "same_srv_rate",
    "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate", "dst_host_rerror_rate",
    "dst_host_srv_rerror_rate", "label"
]

df_train = pd.read_csv("KDDTrain+.txt", names=col_names)
df_test = pd.read_csv("KDDTest+.txt", names=col_names)
```

Ова го вчитува тренинг- и тест-множеството и ги поставува имињата на сите 41+1 колони, бидејќи изворните датотеки немаат заглавје.

4.2. Поделба на тренинг и тест множество

За разлика од вообичаената пракса каде тренинг- и тест-множеството се добиваат со случајна поделба (на пример, преку `train_test_split` во сооднос 80/20), кај NSL-KDD поделбата е однапред дефинирана од самите автори на множеството и доаѓа како две одделни датотеки: KDDTrain+ (околу 125.973 записи) и KDDTest+ (22.544 записи), што одговара на сооднос од приближно 85% наспроти 15% од вкупниот број податоци. Оваа поделба намерно не е случајна: тест-множеството содржи и типови напади што воопшто не се појавуваат во тренинг-множеството, со цел евалуацијата да не мери само меморирање на веќе видени примери, туку вистинска способност моделот да генерализира и врз претходно непознати варијации на напади — услов многу поблизок до реална, оперативна употреба отколку стандардна случајна поделба.

```
df_train["target"] = df_train["label"].apply(lambda x: 0 if x == "normal" else 1)
df_test["target"] = df_test["label"].apply(lambda x: 0 if x == "normal" else 1)

X_train = df_train.drop(columns=["label", "target"])
y_train = df_train["target"]

X_test = df_test.drop(columns=["label", "target"])
y_test = df_test["target"]
```

Колоната label се трансформира во бинарна целна променлива target: „normal“ се пресликува во 0, а секој друг тип напад во 1. За многукласниот режим (делот 5.5), истата колона наместо во 0/1 се пресликува во едно од пет имиња на класи — normal, DoS, Probe, R2L или U2R — преку однапред дефинирана мапа на поединечните напади (Табела 1).

4.3. Претпроцесирање на податоците

4.3.1. Поделба на нумерички и категоријални карактеристики

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

cat_cols = ["protocol_type", "service", "flag"]
num_cols = [col for col in X_train.columns if col not in cat_cols]
```

4.3.2. Претпроцесинг pipeline

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ]
)
```

Ова е најважниот дел од претпроцесирањето — истиот preprocessor автоматски ги обработува и нумеричките и категоријалните колони, а понатаму се вградува во секој од моделите подолу.

4.4. Decision Tree модел

```
from sklearn.tree import DecisionTreeClassifier

pipe_dt = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", DecisionTreeClassifier(random_state=42))
])

pipe_dt.fit(X_train, y_train)
y_pred_dt = pipe_dt.predict(X_test)
```

4.5. Random Forest модел

```
from sklearn.ensemble import RandomForestClassifier

pipe_rf = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", RandomForestClassifier(n_estimators=200, n_jobs=-1,
    random_state=42))
])
```

```

])

pipe_rf.fit(X_train, y_train)
y_pred_rf = pipe_rf.predict(X_test)

```

4.6. Logistic Regression модел

```

from sklearn.linear_model import LogisticRegression

pipe_lr = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", LogisticRegression(max_iter=1000))
])

pipe_lr.fit(X_train, y_train)
y_pred_lr = pipe_lr.predict(X_test)

```

4.7. Neural Network (MLP) модел

```

from sklearn.neural_network import MLPClassifier

pipe_mlp = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=300,
random_state=42))
])

pipe_mlp.fit(X_train, y_train)
y_pred_mlp = pipe_mlp.predict(X_test)

```

4.8. Евалуација на моделите

```

from sklearn.metrics import accuracy_score, classification_report
import joblib, time

def evaluate_and_save(name, pipe, y_pred, out_dir):
    print(f"=== {name} ===")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))
    joblib.dump(pipe, f"{out_dir}/{name}.joblib")    # се зачувува за локална
употреба

```

По тренирањето, секој pipeline (претпроцесирање + модел) се зачувува како единствена joblib-датотека. Со тоа локалното предвидување врз нови податоци се сведува на едноставно вчитување на датотеката и повик на `model.predict(...)`, без потреба рачно да се повторува претпроцесирањето — истата логика на трансформација автоматски важи и при тренирањето, и при подоцнежната примена. Заедно со моделите се зачувуваат и метаподатоци (имиња и редослед на атрибутите, мапирање на класите и точните верзии на користените библиотеки), бидејќи зачуваните модели се чувствителни на верзијата на scikit-learn со која се создадени.

5. Резултати

Перформансите на сите модели се оценети врз посебното тест-множество KDDTest+, кое, како што беше истакнато во делот 4.2, содржи и напади што воопшто не постојат во тренинг-множеството — затоа добиените бројки реално ја одразуваат способноста за генерализација, а не само меморирање. Сите вредности во овој дел се добиени со непосредно вчитување на зачуваните модели и повторно пресметување на метриците врз KDDTest+, со фиксирано случајно семе (random_state=42) за моделите кај кои тоа е применливо, со цел резултатите да бидат репродуцибилни.

5.1. Класификациски резултати (бинарна класификација)

Во ова поглавје се прикажани резултатите добиени од четирите модели на машинско учење обучени за бинарна класификација — нормален сообраќај наспроти напад. За секој модел се прикажани точност (Accuracy), прецизност (Precision), одзив (Recall) и F1-мерка, посебно за класата 0 (нормален сообраќај) и класата 1 (напад).

Модел	Accuracy (точност)	Precision (0/1)	Recall (0/1)	F1-score (0/1)
Decision Tree	0.7854	0.67 / 0.96	0.97 / 0.65	0.80 / 0.77
Random Forest	0.7677	0.66 / 0.97	0.97 / 0.61	0.78 / 0.75
Logistic Regression	0.7537	0.65 / 0.92	0.93 / 0.62	0.76 / 0.74
MLP Neural Network	0.7827	0.68 / 0.93	0.93 / 0.67	0.79 / 0.78

Табела 4. Резултати од бинарната класификација (нормален сообраќај наспроти напад), верификувани со повторно вчитување на зачуваните модели врз KDDTest+. Класата 0 = нормален сообраќај; класата 1 = напад.

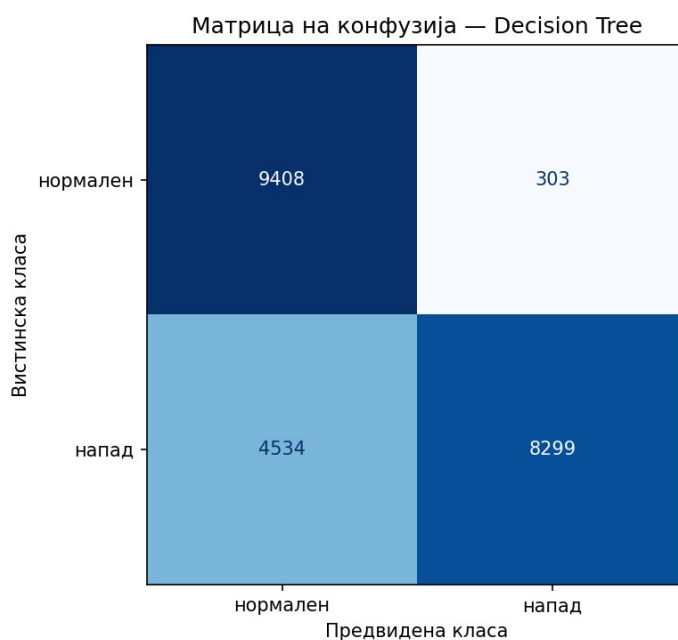
Сите четири модели постигнуваат солидна точност во опсег од околу 75% до 79%, при што дрвото на одлучување и MLP-мрежата се благо пред другите два модели по точност, додека Random Forest се издвојува подоцна, во делот 5.4, како модел со најдобра севкупна моќ на разликување (највисок AUC), и покрај тоа што нема најголема точност. Ова покажува дека самата точност не е доволна мерка — во контекст на IDS-систем поважно е колку добро моделот ги рангира примероците според веројатноста за напад, што токму AUC го мери.

Забележливо е и дека сите модели имаат повисока прецизност за класата „напад“ отколку за класата „нормален сообраќај“, но понизок одзив за нападите — со други зборови, кога моделот означи нешто како напад, најчесто е во право, но пропушта дел од реалните напади (лажно негативни, FN). Токму FN се најкритичната грешка во

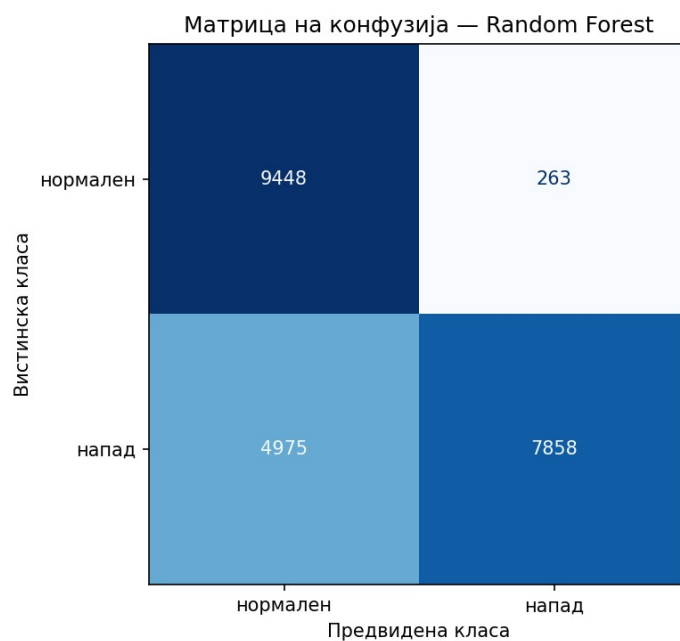
безбедносен контекст, бидејќи значи препознаен напад, и затоа е детално разгледана во делот 5.2.

5.2. Матрица на конфузија

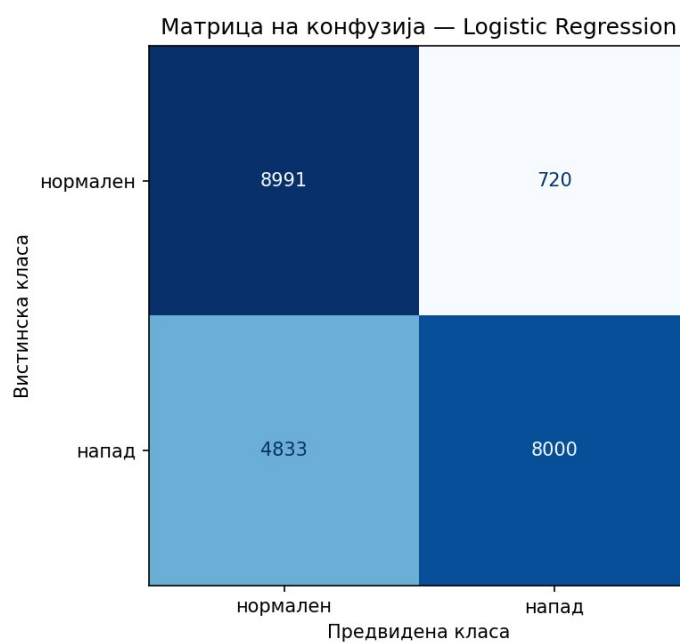
Матрицата на конфузија претставува една од најважните визуелизации при оценување на моделите за класификација, особено во системи за детекција на мрежни напади, бидејќи покажува не само колку примероци се точно класифицирани, туку и во кој правец се греша. Во нашиот случај, класите се: 0 — нормален сообраќај, 1 — напад. Матрицата ги содржи четирите основни категории: точно позитивно (TP) — правилно детектирани напади; точно негативно (TN) — правилно детектиран нормален сообраќај; погрешно позитивно (FP) — нормален сообраќај погрешно класифициран како напад; и погрешно негативно (FN) — напад кој моделот не го препознал, најкритичната грешка во безбедносен систем.



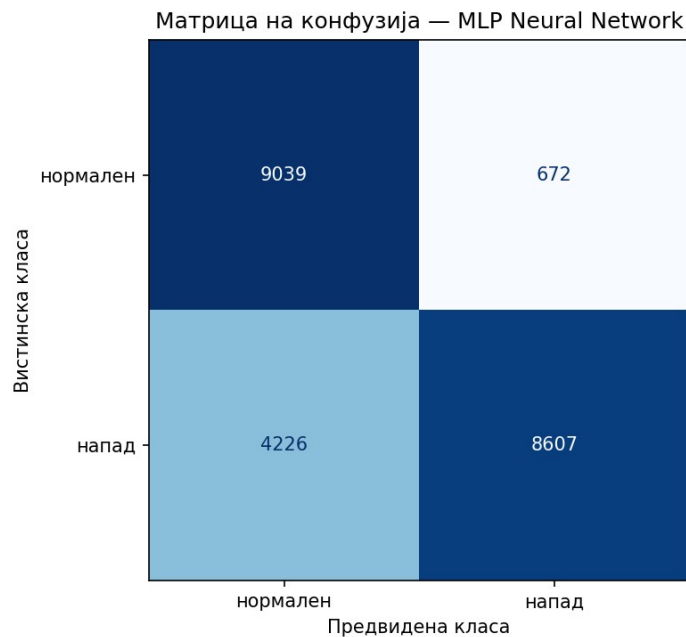
Слика 4. Матрица на конфузија — Decision Tree.



Слика 5. Матрица на конфузија — Random Forest.



Слика 6. Матрица на конфузија — Logistic Regression.



Слика 7. Матрица на конфузија — MLP Neural Network.

Decision Tree добро ги препознава нормалните конекции (висок TN), но пропушта релативно голем дел од нападите (FN). Random Forest се однесува слично, со малку помал број лажно позитивни примери, но со сличен предизвик кај FN. Logistic Regression, како линеарен модел, тешко ги моделира нелинеарните обрасци во сообраќајот, што резултира со највисока комбинација на FP и FN од сите четири модели. MLP Neural Network постигнува најбалансиран однос меѓу TP и TN — најмал број FN и добра детекција на напади — што укажува дека способноста да се научат нелинеарни интеракции меѓу карактеристиките директно се преведува во подобра, побезбедна детекција.

Безбедносна импликација: во контекст на IDS, FN (непрепознаен напад) е значително поризичен од FP (лажен аларм), бидејќи FP само генерира дополнителна проверка, додека FN значи дека вистински напад минал незабележано. Оттука, при избор на модел за реална примена, покрај точноста треба да се земе предвид и одзивот (recall) специфично за класата „напад“, а не само севкупната точност.

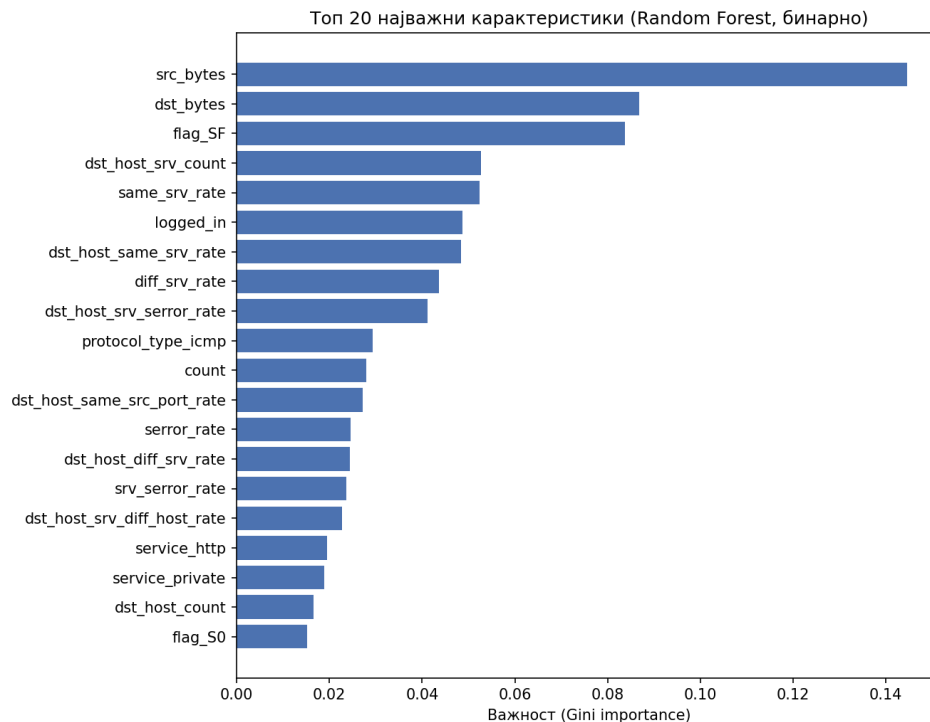
5.3. Важност на карактеристиките (Feature Importance)

```
import matplotlib.pyplot as plt
import numpy as np

rf = pipe_rf.named_steps["clf"]
ohe = pipe_rf.named_steps["preprocessor"].named_transformers_["cat"]
feat_names = num_cols + list(ohe.get_feature_names_out(cat_cols))
```

```
importances = rf.feature_importances_
order = np.argsort(importances)[::-1][:20]

plt.figure(figsize=(9, 7))
plt.barh(np.array(feat_names)[order][::-1], importances[order][::-1])
plt.title("Топ 20 најважни карактеристики")
plt.tight_layout()
plt.show()
```



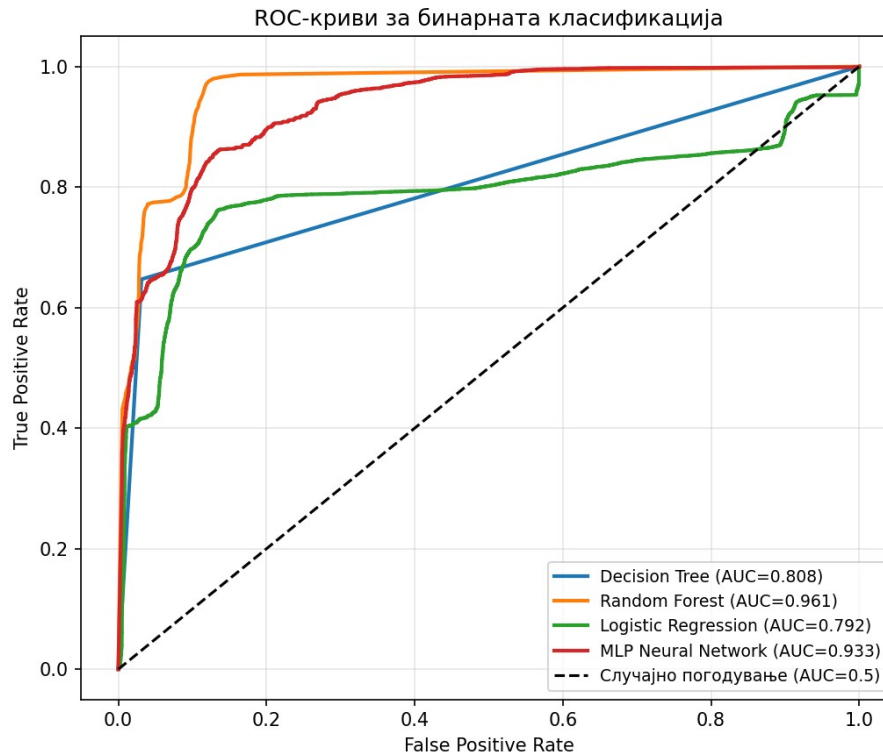
Слика 8. Топ 20 најважни карактеристики според моделот Random Forest (бинарна класификација).

Најзначајни се `src_bytes` и `dst_bytes` (обем на пренесени податоци во двете насоки), `flag_SF` (успешно завршена конекција) и неколку сообраќајни статистики како `dst_host_srv_count` и `same_srv_rate`. Треба да се напомене едно важно ограничување: бидејќи категоријалните атрибути (`protocol_type`, `service`, `flag`) се распространети преку one-hot кодирање, важноста на секоја поединечна категорија (на пример `protocol_type_icmp` или `service_http`) се прикажува одделно, наместо збирно за целиот атрибут. Оттука, вкупниот придонес на атрибутот `service`, на пример, е поголем отколку што изгледа од секоја негова поединечна категорија прикажана на графикот — за целосно точна споредба меѓу изворните атрибути, важностите на сите категории од ист атрибут треба да се сумираат.

5.4. ROC-крива и AUC-анализа

ROC-кривата ја прикажува способноста на моделите да разликуваат помеѓу нормален сообраќај и мрежни напади при различни прагови на одлучување, а AUC (Area Under

the Curve) е единствен број кој ја сумира вкупната моќ на разликување на моделот — колку поблиску до 1, толку подобра способност да ги детектира нападите; вредност од 0.5 одговара на случајно погодување.



Слика 9. ROC-криви за бинарната класификација, со соодветните AUC-вредности, добиени со повторно вчитување на зачуваните модели врз KDDTest+.

Верификуваните AUC-вредности се: Random Forest — 0.961 (најдобар модел според оваа мерка, со одлична разделба меѓу класите); MLP Neural Network — 0.933 (многу добра, блиску до Random Forest); Decision Tree — 0.808 (солидна, но забележливо пониска бидејќи едно единствено дрво дава груба, скалеста ROC-крива); Logistic Regression — 0.792 (најслаба од четирите, но сепак јасно подобра од случајно погодување).

Оваа анализа е добиена со непосредно повторно пресметување врз зачуваните joblib-модели, за да се потврдат бројките пред нивно објавување. Логистичката регресија останува најслаб модел во споредбата, но нејзиниот AUC од 0.792 покажува дека сепак носи разумна, употреблива моќ на разликување — линеарната граница едноставно не е доволно изразителна за целосно да ги фати нелинеарните обрасци во сообраќајот, за разлика од ансамблите од дрва и невронската мрежа.

5.5. Многукласна класификација

Покрај бинарното сценарио, истите четири модели се обучени и во многукласен режим, во кој, наместо само нормално/напад, секој запис се распределува во една од пет класи: normal, DoS, Probe, R2L или U2R (согласно групирањето од Табела 1). Ова овозможува подетална слика за природата на секоја закана, наместо само бинарен сигнал за тревога.

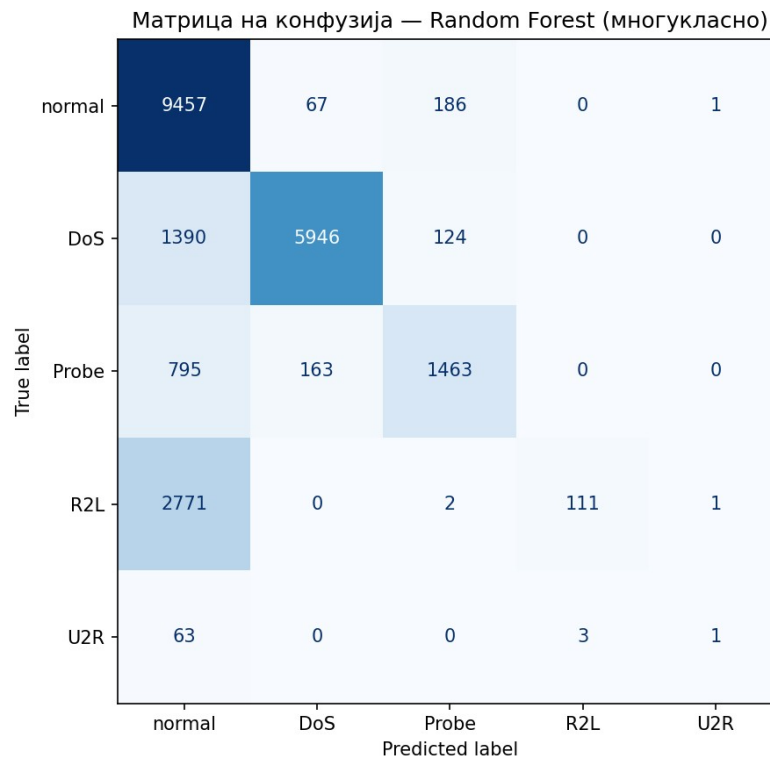
Модел	Accuracy (точност)	macro-F1	Време на тренирање (s)
Decision Tree	0.7651	0.5739	4.2
Random Forest	0.7531	0.4908	35.9
Logistic Regression	0.7614	0.5692	40.2
MLP Neural Network	0.7637	0.5721	150.6

Табела 5. Резултати од многукласната класификација (пет класи) врз *KDDTest+*.

Вкупната точност во многукласниот режим останува во сличен опсег како и во бинарниот, но збирната мерка macro-F1 е значително пониска кај повеќето модели. Причината станува јасна штом резултатите се разложат по класи, како во Табела 6, прикажана за најдобриот модел според macro-F1 меѓу ансамблите — Random Forest:

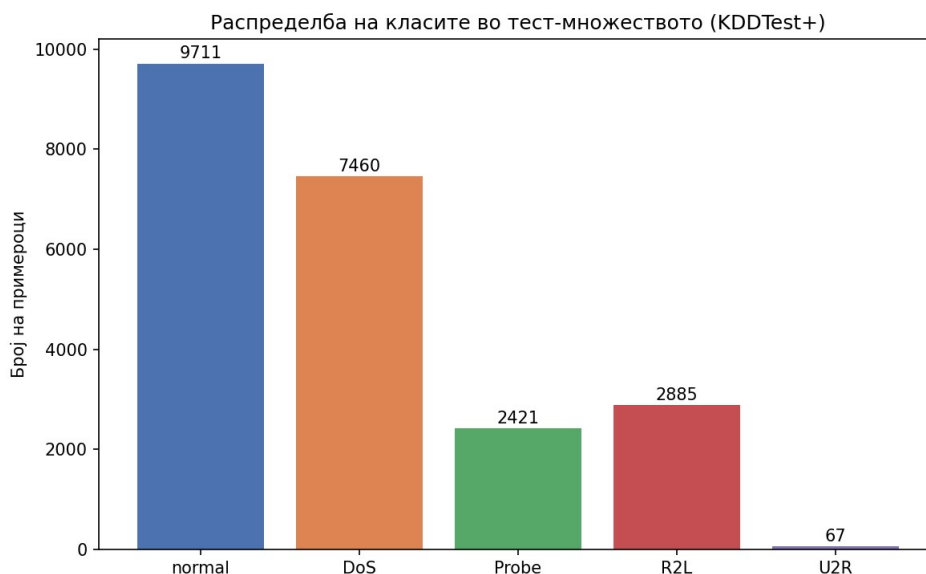
Класа	Precision	Recall	F1	Примери
DoS	0.963	0.797	0.872	7460
Probe	0.824	0.604	0.697	2421
R2L	0.974	0.039	0.074	2885
U2R	0.333	0.015	0.029	67
normal	0.653	0.974	0.782	9711

Табела 6. Точност по класа за моделот *Random Forest* во многукласното сценарио.



Слика 10. Матрица на конфузија — Random Forest (многукласно).

Честите фамилии DoS и Probe се препознаваат добро, со F1 над 0.69. Ретките класи R2L и U2R остануваат голем предизвик: моделот честопати постигнува висока прецизност за нив (кога веќе означи примерок како R2L, најчесто е во право), но неговиот одзив е многу низок — ги пропушта најголемиот дел од тие напади, класифицирајќи ги наместо тоа како нормален сообраќај (види ја колоната „normal“ во Слика 10). Ова директно произлегува од силната нерамнотежа на класите, прикажана на Слика 1: со само 67 примери за U2R и 2885 за R2L во целото тест-множество (наспроти 7460 за DoS и 9711 за normal), моделот нема доволно основа да ги научи нивните ретки, но опасни обрасци. Станува збор за добро познато ограничување токму на ова множество, а не за грешка во пристапот — и токму затоа балансирањето на класите (SMOTE или класно-тежинско пондерирање) е предложено како приоритетна насока за понатамошна работа во делот 7.5.



Слика 1. Распределба на класите во тест-множеството (KDDTest+) — јасно се гледа нерамнотежата меѓу честите класи (normal, DoS) и ретките класи (R2L, U2R).

5.6. Пресметковна сложеност

За еден IDS-систем не е важна само точноста туку и брзината — колку време е потребно за тренирање, а особено колку брзо моделот носи одлука за нова конекција, бидејќи тоа директно ја определува можноста за примена во реално време. Табела 7 ги сумира измерените времиња на тренирање за сите модели, во двата режима, на иста машина и исто множество податоци.

Модел	Време — бинарно (s)	Време — многукласно (s)
Decision Tree	10.8	4.2
Random Forest	46.6	35.9
Logistic Regression	13.1	40.2
MLP Neural Network	141.1	150.6

Табела 7. Пресметковна сложеност — времиња на тренирање (во секунди) за секој модел и режим.

Decision Tree е убедливо најбрз за тренирање, а Random Forest бара умерено повеќе време поради големиот број дрва во ансамблот. MLP Neural Network е највремено барателен модел, до 10–15 пати побавен од дрвото на одлучување, бидејќи тренирањето на невронска мрежа вклучува итеративна оптимизација низ повеќе епохи. Важно е да се напомене дека времето на тренирање се троши еднократно, во облакот (делот б), додека времето на предвидување за една единствена нова конекција — релевантно за реално-времена примена — е за сите четири модели практично моментално (под

неколку милисекунди), бидејќи станува збор само за проследување низ веќе научен pipeline, без каква било повторна оптимизација.

6. Архитектура на решението и локална примена

Решението е замислено во две фази што природно одговараат на реален работен тек: фаза на тренирање во облак и фаза на употреба на локална машина. Тренирањето се изведува во Google Colab [5], која нуди бесплатни пресметковни ресурси и веднаш подготвена околина, погодна за обучување повеќе модели врз релативно обемно множество податоци. Откако ќе завршат сите модели, тие — заедно со претпроцесирањето и придружните метаподатоци — се зачувуваат и се пакуваат во архива (nsl_kdd_models.zip) која потоа лесно се презема, како што е прикажано на Слика 2.



Слика 2. Работен тек на решението: тренирање и зачувување во Google Colab (Фаза 1), преземање на архивата и локално вчитување и тестирање врз нов датасет (Фаза 2).

Втората фаза се одвива целосно локално и не бара повторно тренирање. Архивата се распакува на корисничката машина, се вчитува саканиот модел и тој се применува врз нов датасет за да даде предвидувања и, доколку новите податоци содржат ознаки, мерки за точност. Ваквата поделба ги одвојува скапите пресметки (кои се прават еднаш) од честата, евтина употреба на готовиот модел, што е суштината на барањето моделите да можат повторно да се користат.

Локалната употреба ја обезбедува посебна скрипта (`predict_local.py`), дизајнирана да биде толерантна на разновидни влезни податоци: автоматски препознава дали датасетот има заглавје, се справува со присуство или отсуство на ознаките, ги пополнува евентуалните недостасувачки атрибути и предупредува за непознати вредности во категоријалните колони, наместо тивко да даде погрешен резултат. Излезот вклучува табела со точност по модел, датотека `predictions.csv` со предвидувањата и матрица на конфузија за секој модел.

За практична демонстрација на системот е изградена и едноставна веб-апликација со Gradio (`app_gradio.py`), која ги користи веќе истренираните модели без повторно тренирање. Апликацијата нуди избор на режим (бинарна или многукласна класификација) и модел (Decision Tree, Random Forest, Logistic Regression или MLP), анализа на цел датасет со прикажување точност, распределба на предвидувања и матрица на конфузија, како и тестирање една единствена мрежна конекција со јасна пресуда „напад“ или „нормално“, со непосредна споредба со вистинската ознака. На тој начин, целиот процес на детекција — истиот што е теоретски опишан во делот 3.6 и прикажан на Слика 3 — може реално да се набљудува во дејство.

7. Заклучок

Во рамки на овој проект беше изграден функционален систем за детекција на мрежни напади (Intrusion Detection System), базиран на техники од вештачка интелигенција, користејќи го множеството податоци NSL-KDD. Податоците беа обработени со прецизни техники за претпроцесирање, вклучувајќи стандардизација, one-hot кодирање и дефинирање на целната променлива, а целата логика е спакувана во единствен, повторно употреблив pipeline.

Беа имплементирани и споредени четири различни модели — Decision Tree, Random Forest, Logistic Regression и MLP Neural Network — во два сценарија: бинарна класификација (нормално/напад) и многукласна класификација (пет фамилии на сообраќај). Верификуваната анализа покажа дека Random Forest е најдобар модел според AUC (0.961), што укажува на извонредна способност за разликување нормален од нападнат сообраќај, а MLP Neural Network постигна многу близок резултат (AUC = 0.933) со добар баланс помеѓу прецизност и одзив. Decision Tree даде солидни, брзи за тренирање резултати, но со поголем број лажно негативни детекции, додека Logistic Regression, иако најслаб од четирите модели (AUC = 0.792), сепак покажа употреблива, а не случајна моќ на разликување.

Многукласната класификација дополнително покажа дека честите категории напади (DoS, Probe) се препознаваат добро, но ретките, а притоа многу опасни категории (R2L, U2R) остануваат голем предизвик поради нивниот мал број примери во множеството — наод што директно ја мотивира потребата од техники за балансирање на податоците во понатамошна работа.

Практичната импликација на овие резултати е директна: во сајбер-безбедноста, пропуштен напад (лажно негативен резултат, FN) е значително поскап и поризичен настан од лажен аларм (лажно позитивен резултат, FP) — FP предизвикува само дополнителна, рутинска проверка, додека FN значи дека вистински напад останал незабележан и може да предизвика реална штета. Затоа изборот на модел за евентуална реална примена треба првенствено да се води според одзивот (recall) кон класата „напад“ и способноста за рано откривање ретки, но опасни категории напади, а не само според севкупната точност.

Конечно, проектот покажа дека архитектура во две фази — тренирање во облак и повторна употреба на зачуваните модели локално, без потреба од повторно тренирање — е практично изводлива и го доближува решението до реална, оперативна употреба во системи за сајбер-безбедност.

8. Идни подобрувања

Иако проектот веќе опфаќа и бинарна и многукласна класификација, постојат повеќе реални насоки за понатамошно продлабочување на работата.

8.1. Напредна оптимизација на хиперпараметри

Моделите во овој проект се тренирани со стандардни/подразбирани хиперпараметри, како појдовна точка за објективна споредба. Систематско пребарување со GridSearchCV или RandomizedSearchCV — на пример длабочината и бројот на дрва кај Random Forest, или стапката на учење и архитектурата на слоевите кај MLP — веројатно би довело до повисока точност и подобар баланс помеѓу FP и FN.

8.2. Балансирање на множеството податоци

Како што покажа анализата во делот 5.5, ретките класи R2L и U2R остануваат тешки за детекција токму поради малиот број примери. Примена на техники како SMOTE (синтетичко надополнување на малцинските класи) или класно-тежинско пондерирање (class_weight) кај моделите би можело значително да го подобри одзивот кон овие ретки, но опасни категории напади.

8.3. Употреба на длабоки невронски мрежи

Проектот веќе вклучува една подлабока Keras-мрежа како истражувачки додаток (делот 3.4). Понатамошна работа може да ги опфати и специјализирани архитектури како 1D конволуциски невронски мрежи (CNN) или рекурентни мрежи (LSTM/GRU), кои во литературата често даваат врвни резултати во IDS-системи, како и автоенкодери за детекција на аномалии без потреба од означени примери на напади.

8.4. Реално-временска детекција

Интеграција со мрежен интерфејс (на пример преку Scapy или tcpdump) би овозможила читање пакети во реално време, нивна брза екстракција на карактеристики и онлајн предвидување со веќе истренираните модели — чекор кој практично би го трансформирал проектот во работен IDS-прототип, надоврзувајќи се директно на веќе постоечката Gradio-демонстрација (делот 6).

8.5. Обука и тестирање со понови множества податоци

NSL-KDD е класичен и добро документиран, но не го одразува целосно современиот мрежен сообраќај. Природна насока е истите техники на претпроцесирање и

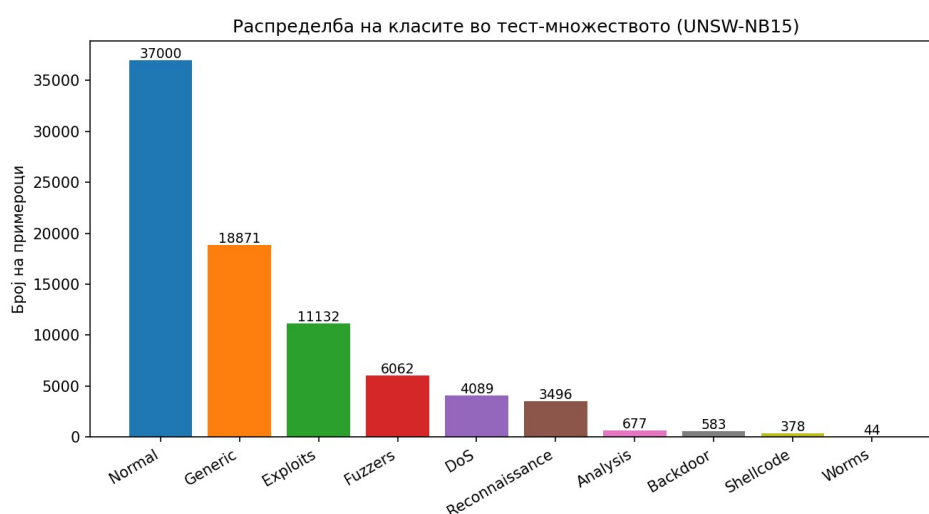
моделирање да се применат и врз понови множества, како CIC-IDS 2017, UNSW-NB15 или CIDDS-001, кои содржат посовремени типови напади и сообраќај. Важно е да се напомене дека овие множества имаат поинаква шема на атрибути од NSL-KDD, па директна повторна употреба на веќе истренираните модели не е возможна — потребно е ново, посебно тренирање со ист методолошки пристап. Овој чекор веќе е реализиран за UNSW-NB15 и е детално опишан во делот 9; CIC-IDS 2017 и CIDDS-001 остануваат отворени насоки за понатамошна валидација.

9. Валидација на пристапот врз друго множество податоци (UNSW-NB15)

Според забелешката на менторот во делот 8.5, истата методологија — ист pipeline на претпроцесирање и истите четири модели — е применета и врз друго, поново множество податоци: UNSW-NB15 [6]. Ова множество е создадено во Australian Centre for Cyber Security (ACCS) со комбинација на реален нормален сообраќај и синтетички генерирани современи напади, со карактеристики извлечени преку алатките Argus и Bro-IDS (сега Zeek).

9.1. Опис на UNSW-NB15

Како и кај NSL-KDD, множеството доаѓа со официјална, однапред дефинирана поделба: UNSW_NB15_training-set (175.341 записи) и UNSW_NB15_testing-set (82.332 записи). Секој запис содржи 34 карактеристики (по исклучување на целните колони) — 31 нумерички и 3 категоријални (proto, service, state) — наспроти 41-те атрибути кај NSL-KDD, при што proto самиот содржи 133 различни вредности на мрежни протоколи, значително повеќе од трите вредности кај protocol_type во NSL-KDD. Целната класа за бинарна класификација (label) е веќе дадена како 0/1 (нормално/напад), а за многукласна класификација множеството директно дава колона attack_cat со девет фамилии напади — Generic, Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor, Shellcode и Worms — покрај класата Normal, наместо четирите фамилии кај NSL-KDD.



Слика 11. Распределба на класите во тест-множеството на UNSW-NB15. Забележлива е слична, но поинаква нерамнотежа од онаа кај NSL-KDD — класата Worms има само 44 примери во тест-множеството.

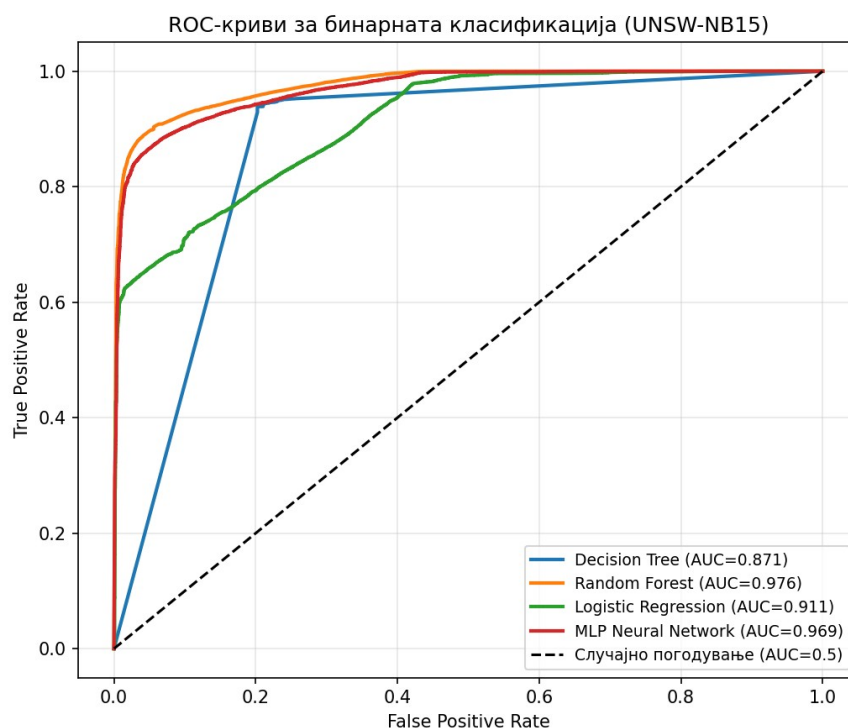
9.2. Прилагодување на pipeline-от

Прилагодувањето беше минимално, што ја потврдува предноста на pipeline-пристапот опишан во делот 3.3 и 4.3: истиот ColumnTransformer (StandardScaler за нумеричките атрибути, OneHotEncoder(handle_unknown='ignore') за категоријалните) и истите четири модели (со идентични хиперпараметри како во делот 4.4–4.7) се применети без промена на архитектурата — единствената измена е листата на имиња на колони (proto/service/state наместо protocol_type/service/flag) и директната достапност на веќе готова целна колона за секој режим, без потреба од дополнително пресликување.

9.3. Резултати

Модел	Accuracy	Precision (0/1)	Recall (0/1)	F1-score (0/1)
Decision Tree	0.8664	0.93 / 0.83	0.76 / 0.95	0.84 / 0.89
Random Forest	0.8656	0.96 / 0.82	0.73 / 0.97	0.83 / 0.89
Logistic Regression	0.7969	0.94 / 0.74	0.59 / 0.97	0.72 / 0.84
MLP Neural Network	0.8533	0.95 / 0.81	0.71 / 0.97	0.81 / 0.88

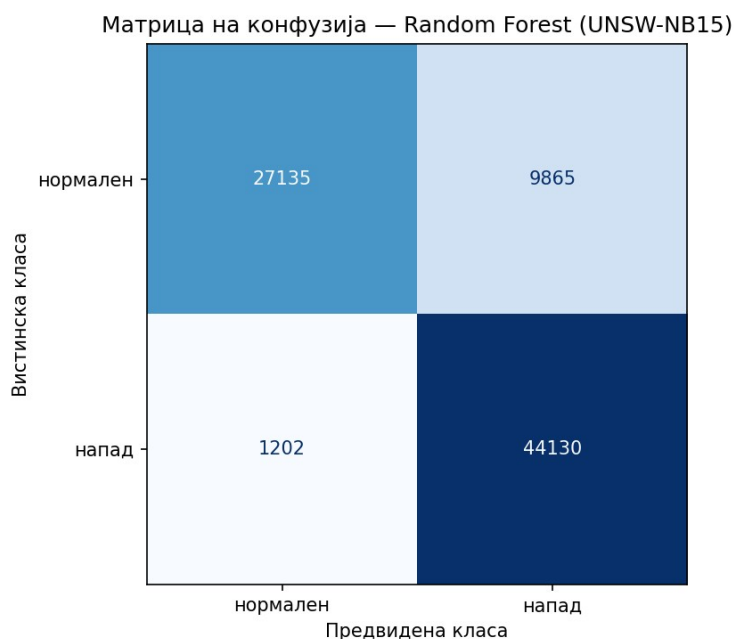
Табела 8. Резултати од бинарната класификација врз UNSW-NB15 (0 = нормален сообраќај, 1 = напад).



Слика 12. ROC-криви за бинарната класификација врз UNSW-NB15.

AUC-вредностите на UNSW-NB15 се: Random Forest — 0.976, MLP Neural Network — 0.969, Logistic Regression — 0.911 и Decision Tree — 0.871. Забележливо е дека сите

модели постигнуваат повисок AUC отколку врз NSL-KDD (делот 5.4), а особено логистичката регресија — нејзиниот AUC скока од 0.792 на 0.911. Ова укажува дека карактеристиките во UNSW-NB15 (изведени преку Argus/Bro, вклучувајќи статистики како sload, dload, sinpkt) обезбедуваат полинеарно раздвоива граница меѓу нормален и нападнат сообраќај, за разлика од понекомплексните односи во NSL-KDD.



Слика 13. Матрица на конфузија — Random Forest, бинарно (UNSW-NB15).

Многукласна класификација

Модел	Accuracy	macro-F1	Време на тренирање (s)
Decision Tree	0.7478	0.4834	16.9
Random Forest	0.7696	0.4537	108.8
Logistic Regression	0.6712	0.3190	57.3
MLP Neural Network	0.7754	0.4599	822.0

Табела 9. Резултати од многукласната класификација (десет класи) врз UNSW-NB15.

Класа	Precision	Recall	F1	Примери
Normal	0.943	0.781	0.854	37000
Generic	0.998	0.968	0.983	18871
Exploits	0.588	0.842	0.692	11132
Fuzzers	0.304	0.554	0.393	6062
DoS	0.496	0.107	0.177	4089
Reconnaissance	0.932	0.799	0.860	3496
Analysis	0.004	0.003	0.004	677

Класа	Precision	Recall	F1	Примери
Backdoor	0.026	0.072	0.038	583
Shellcode	0.445	0.479	0.461	378
Worms	0.222	0.045	0.076	44

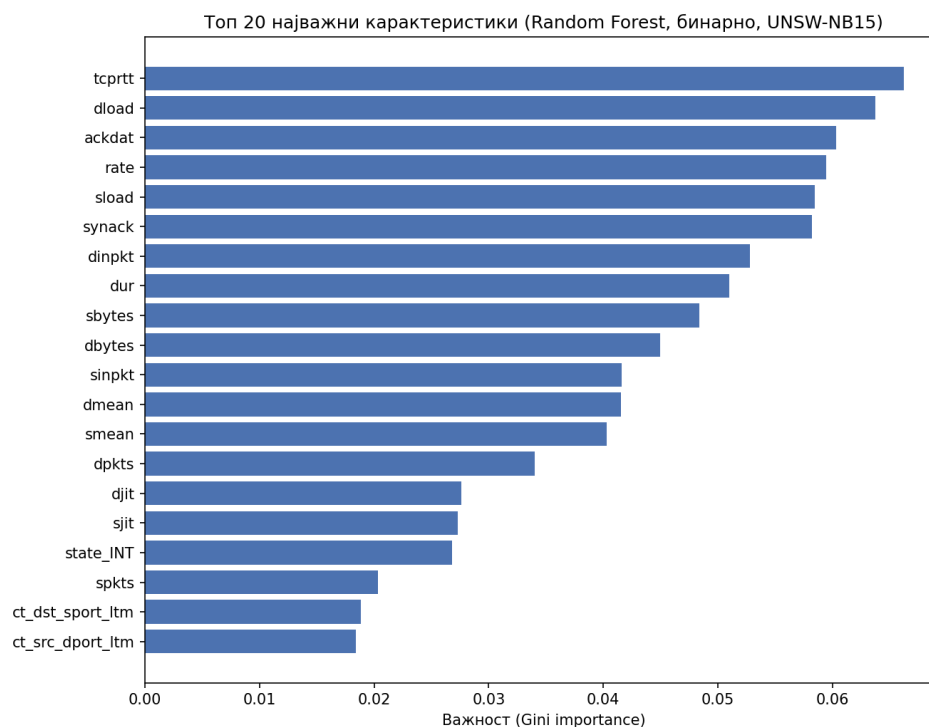
Табела 10. Точност по класа за моделот *Random Forest* во многукласното сценарио (*UNSW-NB15*).

Матрица на конфузија — *Random Forest*, многукласно (*UNSW-NB15*)

True label \ Predicted label	Normal	Generic	Exploits	Fuzzers	DoS	Reconnaissance	Analysis	Backdoor	Shellcode	Worms
Normal	28897	18	655	6921	37	9	357	0	103	3
Generic	34	18270	463	64	19	1	0	4	14	2
Exploits	280	8	9378	374	207	156	56	634	38	1
Fuzzers	1270	4	1202	3357	85	7	7	100	29	1
DoS	65	8	2658	160	439	30	32	673	24	0
Reconnaissance	42	1	474	55	21	2793	6	88	16	0
Analysis	1	0	554	1	36	0	2	83	0	0
Backdoor	1	0	502	4	30	0	2	42	2	0
Shellcode	51	0	39	95	10	2	0	0	181	0
Worms	1	1	35	3	2	0	0	0	0	2

Слика 14. Матрица на конфузија — *Random Forest*, многукласно (*UNSW-NB15*).

Резултатите потврдуваат два наода веќе видени кај *NSL-KDD* (делот 5.5). Прво, силно застапените класи (*Generic*, *Normal*, *Reconnaissance*) се препознаваат одлично, додека најретките (*Worms* — само 44 примери во тестот, *Analysis*, *Backdoor*) речиси и не се детектираат — истата причина, недоволен број тренинг-примери. Второ, се појавува и специфичен проблем на самото *UNSW-NB15*: категориите *DoS* и *Exploits* се системски мешаат меѓу себе (2658 од 4089 примери на *DoS* се предвидени како *Exploits*, види Табела 10 и Слика 14) — познато ограничување на ова множество, коментирано и во оригиналната литература [6], бидејќи двете категории напади содржат преклопувачки, слично изгледни мрежни обрасци.



Слика 15. Топ 20 најважни карактеристики според Random Forest (бинарно, UNSW-NB15).

Најважните карактеристики за детекција на напади во UNSW-NB15 целосно се разликуваат од NSL-KDD — доминираат карактеристиките поврзани со TCP-конекцијата (tcprrt, dload, ackdat, rate, sload, synack — статистики за времето на воспоставување и текот на TCP-врската), додека обемот на податоци (sbytes/dbytes) и категоријалната state_INT се присутни, но со помал придонес. Кај NSL-KDD, пак, доминираа токму обемот на податоци (src_bytes/dst_bytes) и статусот на конекцијата (flag_SF). Ова покажува дека, иако методологијата е иста, конкретните индикатори на напад зависат од доменот и начинот на собирање на податоците.

9.4. Споредба со NSL-KDD и заклучок од валидацијата

Мерка	NSL-KDD	UNSW-NB15
Најдобар модел (бинарно, по AUC)	Random Forest (0.961)	Random Forest (0.976)
Logistic Regression AUC	0.792	0.911
Точност — бинарно (опсег)	0.75 – 0.79	0.80 – 0.87
Најголем предизвик — многукласно	R2L, U2R (петки класи)	Worms, Analysis, Backdoor (петки класи) + DoS/Exploits преклопување
Број класи (многукласно)	5	10
Категоријални атрибути	protocol_type, service, flag	proto (133 вредности), service, state

Табела 11. Директна споредба на клучните наоди меѓу NSL-KDD и UNSW-NB15.

Валидацијата потврдува дека методологијата развиена во овој проект — единствен ColumnTransformer pipeline и истите четири модели — е директно пренослива врз друго множество податоци, без промена на архитектурата, само со прилагодување на имињата на колоните. Апсолутните резултати се повисоки на UNSW-NB15, но квалитативните заклучоци остануваат идентични на оние од NSL-KDD: ансамбл-моделите (Random Forest) и невронската мрежа (MLP) се најсоодветни за бинарна детекција, додека силната нерамнотежа на класите останува главната пречка за успешна многукласна класификација на ретките, но опасни категории напади — наод кој не зависи од конкретното множество податоци, туку е фундаментално својство на проблемот на детекција на мрежни напади.

10. ЛИТЕРАТУРА

- [1] Tavallaei, M., Bagheri, E., Lu, W., Ghorbani, A. A., "A Detailed Analysis of the KDD CUP 99 Data Set," in Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009. (изворен труд во кој е претставено множеството NSL-KDD)
- [2] Buczak, A. L., Guven, E., "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.
- [3] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] Chollet, F. et al., "Keras," <https://keras.io>, 2015.
- [5] Google LLC, "Google Colaboratory Documentation," <https://colab.research.google.com>, пристапено 2025/2026.
- [6] Moustafa, N., Slay, J., "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in Proc. Military Communications and Information Systems Conference (MilCIS), IEEE, 2015. (изворен труд во кој е претставено множеството UNSW-NB15, користено во делот 9)

Забелешка: при подготовката на кодот и оваа документација беа користени и алатки со вештачка интелигенција (Google Colab со вградени AI-помагала и Claude), исклучиво за помош при пишување и рефакторирање на кодот и текстот; сите експериментални резултати во деловите 5 и 9 се верификувани со непосредно повторно извршување врз зачуваните модели.

11. Прилози

Прилог 1: Google Colab notebook со имплементација на алгоритмот врз NSL-KDD (IDS_train.ipynb) —

https://colab.research.google.com/drive/1J3fvS5g2yDDZOQGwIMZ1q_GtnaJD584b?usp=sharing

Прилог 2: Целосниот изворен код за NSL-KDD, зачуваните модели (models/binary, models/multiclass), тест-множеството (KDDTest+.txt), скриптата за локално предвидување (predict_local.py) и демонстрационата Gradio-апликација (app_gradio.py) се приложени во придружната папка на проектот, во потпапката code/.

Прилог 3: Извештаите за секој модел врз NSL-KDD (accuracy, precision, recall, F1, време на тренирање) се достапни како текстуални датотеки во code/models/binary/*_report.txt и code/models/multiclass/*_report.txt, заедно со збирната табела code/models/summary.csv.

Прилог 4: Валидацијата врз UNSW-NB15 (делот 9) — скриптата за тренирање (train_unsw.py), зачуваните модели, извештаите и метаподатоците се приложени во потпапката code_unsw/models/, а скриптата за генерирање на графициите во code_unsw/gen_figures_unsw.py.